

OS Unit II - Process Management

Unit II – Process Management

Operating System | MSBTE K-Scheme | Diploma CO Branch | TY

2.1 Processes

What is a Process?

A **process** is a program in execution. A program is a passive entity (just code stored on disk), while a process is an active entity — it has a program counter, a set of resources (CPU, memory, I/O), and a current activity.

Program vs Process: | Program | Process | |—|—| | Passive entity, stored on disk | Active entity, in main memory | | Static | Dynamic, changes state | | Does not need resources | Needs CPU, memory, I/O | | One program can create many processes | Each process is a separate execution instance |

A process in memory typically has four sections: - **Text section** – the executable code - **Data section** – global variables - **Heap** – memory allocated dynamically during run time - **Stack** – temporary data such as function parameters, return addresses, local variables

2.1.1 Process States

As a process executes, it changes **state**. Each process may be in one of the following states:

1. **New** – The process is being created.
2. **Ready** – The process is waiting to be assigned to a processor. It is loaded into main memory and waiting in the ready queue.
3. **Running** – Instructions are being executed by the CPU.
4. **Waiting (Blocked)** – The process is waiting for some event to occur (I/O completion, signal, resource).
5. **Terminated** – The process has finished execution.

State transition diagram (described):

```
New --admit--> Ready --scheduler dispatch--> Running
Running --interrupt (time out)--> Ready
Running --I/O or event wait--> Waiting
Waiting --I/O or event completion--> Ready
Running --exit--> Terminated
```

- Only **one process** can be in the Running state per CPU core at a given time.
 - Many processes may be in the Ready or Waiting state simultaneously.
-

2.1.2 Process Control Block (PCB)

The **PCB** (also called Task Control Block) is a data structure maintained by the OS for every process. It stores all the information needed to track and manage a process. It acts as the “identity card” of a process.

Contents of a PCB: 1. **Process State** – new, ready, running, waiting, terminated. 2. **Process ID (PID)** – unique number identifying the process; also stores Parent Process ID (PPID). 3. **Program Counter (PC)** – address of the next instruction to be executed. 4. **CPU Registers** – accumulator, index registers, stack pointer, general-purpose registers (saved when process is switched out). 5. **CPU Scheduling Information** – process priority, pointers to scheduling queues. 6. **Memory Management Information** – base/limit

registers, page tables or segment tables. 7. **Accounting Information** – CPU time used, time limits, process number. 8. **I/O Status Information** – list of I/O devices allocated, list of open files.

Diagram (described):

```
-----  
| Process State      |  
| Process Number (PID) |  
| Program Counter    |  
| CPU Registers      |  
| Memory Limits      |  
| List of Open Files  |  
| ...                |  
-----
```

Importance: The PCB is created when a process is created and deleted when the process terminates. It is used during **context switching** to save and restore process information so the process can resume exactly where it left off.

2.2 Process Scheduling

The OS must share the CPU among multiple processes to maximize CPU utilization. This is done through **process scheduling**.

2.2.1 Scheduling Queues

As processes enter the system, they are put into a **job queue**. The OS maintains several queues:

1. **Job Queue** – contains all processes in the system (in memory or waiting to enter).
2. **Ready Queue** – contains processes residing in main memory, ready and waiting to execute. Implemented as a linked list; the header holds pointers to the first and last PCBs.
3. **Device Queue (I/O Queue)** – contains processes waiting for a particular I/O device. Each device has its own queue.

Queuing diagram (described): A new process enters the **Ready Queue**. It is dispatched to the CPU. During execution it may: - Issue an I/O request → moves to a **Device Queue** - Create a child process → waits for its termination - Be interrupted → sent back to the Ready Queue - Finish execution → exits the system

This is often represented as a **queueing diagram** with the Ready Queue and multiple Device Queues connected by arrows showing process flow.

2.2.2 Types of Schedulers

Schedulers are OS components responsible for selecting processes for execution. There are three types:

1. **Long-Term Scheduler (Job Scheduler)** - Selects processes from the job pool (on disk) and loads them into memory for execution. - Controls the **degree of multiprogramming** (number of processes in memory). - Executes less frequently (minutes). - Selects a good mix of I/O-bound and CPU-bound processes.
2. **Short-Term Scheduler (CPU Scheduler)** - Selects a process from the Ready Queue and allocates the CPU to it. - Executes very frequently (milliseconds) → must be fast. - Also called CPU scheduler.
3. **Medium-Term Scheduler** - Temporarily removes a process from memory (swapping out) to reduce the degree of multiprogramming, and later reintroduces it (swapping in). - Improves the process mix and frees memory. - Helpful when the system is overloaded.

Comparison Table: | Scheduler | Speed | Function | |—|—|—| | Long-term | Slow (minutes) | Job → Ready queue (controls degree of multiprogramming) | | Short-term | Fast (milliseconds) | Ready → Running (CPU allocation) | | Medium-term | Medium | Swapping in/out of memory |

2.2.3 Context Switch

A **context switch** occurs when the CPU switches from executing one process to another.

Process: 1. The state of the currently running process (its context: register values, program counter, memory info) is saved into its PCB. 2. The state of the next process to run is loaded from its PCB. 3. Control is transferred to the new process, which resumes from where it left off.

When does context switching occur? - When a process is interrupted (e.g., time slice expires in Round Robin) - When a higher-priority process arrives - When a process moves from Running to Waiting (I/O request) - During interrupt handling

Key points: - Context switch is pure **overhead** — the system does no useful work while switching. - Time taken depends on hardware support (some CPUs have multiple register sets to speed this up). - Frequent context switching reduces CPU efficiency, so it should be minimized.

2.3 Inter Process Communication (IPC)

Processes executing concurrently in the OS may be **independent** or **cooperating**. Cooperating processes need a mechanism to exchange data — this is called **Inter Process Communication (IPC)**.

Why IPC is needed: information sharing, computation speedup, modularity, and convenience.

There are two fundamental models of IPC:

2.3.1 Shared Memory System

- A region of memory is shared between cooperating processes.
- Processes can exchange information by reading and writing data to this shared region.
- The OS's role is minimal — it just establishes the shared memory region; **synchronization is the responsibility of the processes** themselves (using semaphores, mutex locks, etc. to avoid race conditions).

Example: Producer-Consumer Problem - A **producer** process produces data/items and places them into a shared buffer. - A **consumer** process consumes items from the shared buffer. - Requires synchronization so the producer doesn't overwrite unconsumed data and the consumer doesn't read empty slots. - Two buffer types: **unbounded buffer** (no size limit) and **bounded buffer** (fixed size).

Advantages: - Faster communication (no kernel intervention needed after setup) since it's just memory access.

Disadvantages: - Synchronization must be handled manually by programmer — more complex and error-prone.

2.3.2 Message Passing System

- Processes communicate by exchanging **messages** without sharing any memory region.
- Useful in distributed systems where processes may reside on different machines.

Basic operations: - `send(message)` - `receive(message)`

Communication requires a link between processes, established via:

1. **Direct Communication** – processes name each other explicitly.
 - `send(P, message)` – send to process P
 - `receive(Q, message)` – receive from process Q
 - Link is established automatically between every pair of processes that want to communicate.
2. **Indirect Communication** – messages sent to and received from **mailboxes (or ports)**.
 - `send(A, message)` – send to mailbox A
 - `receive(A, message)` – receive from mailbox A
 - A link exists between a pair of processes only if they share a mailbox.

Synchronization in message passing: - **Blocking (synchronous)** send/receive – sender/receiver waits until the message is delivered/received. - **Non-blocking (asynchronous)** send/receive – sender continues without waiting.

Buffering (queue capacity): - **Zero capacity** – no messages can wait; sender must block until receiver gets it. - **Bounded capacity** – queue has finite length n . - **Unbounded capacity** – queue length is infinite.

Shared Memory vs Message Passing: | Shared Memory | Message Passing | |—|—| | Faster (after setup) | Slower (kernel involved in every call) | | Used for large amounts of data | Better for small amounts of data | | Synchronization handled by processes | Synchronization handled by OS | | Good for processes on the same machine | Good for distributed systems too |

2.4 Threads

A **thread** is the basic unit of CPU utilization. It is a lightweight process consisting of a thread ID, program counter, register set, and stack. It shares its code section, data section, and OS resources (like open files) with other threads belonging to the same process.

A traditional (heavyweight) process has a single thread of control. Modern OSes support **multithreading**, where a process can have multiple threads performing multiple tasks at the same time.

2.4.1 Benefits of Multithreading

1. **Responsiveness** – An application remains responsive to the user even if part of it is blocked (e.g., a web browser can keep loading images while responding to user clicks).
 2. **Resource Sharing** – Threads share the memory and resources of the process they belong to, so an application can have several threads within the same address space.
 3. **Economy** – Creating and context-switching threads is cheaper than doing so for processes, since threads share resources of the parent process.
 4. **Scalability (Utilization of multiprocessor architecture)** – Threads can run in parallel on different CPU cores, increasing concurrency and improving performance on multi-core systems.
-

2.4.2 User-Level Threads and Kernel-Level Threads

User-Level Threads (ULT): - Managed entirely by a user-level thread library (no kernel involvement). - Thread creation, scheduling, and management are done in user space, so they are fast. - The kernel is **not aware** of user-level threads — it sees only a single process. - **Disadvantage:** If one thread makes a blocking system call, the entire process (all its threads) blocks.

Kernel-Level Threads (KLT): - Managed directly by the operating system kernel. - The kernel performs thread creation, scheduling, and management. - Slower to create/manage than ULTs (requires system calls) but more robust. - **Advantage:** If one thread blocks, the kernel can schedule another thread of the same process — the whole process does not block.

Comparison Table: | User-Level Threads | Kernel-Level Threads | |—|—| | Managed by user-level library | Managed by OS kernel | | Fast to create and manage | Slower (system call overhead) | | Kernel unaware of threads | Kernel aware of each thread | | One blocking thread blocks entire process | Other threads can continue if one blocks | | Example: POSIX Pthreads (user library) | Example: Windows, Linux native threads |

2.4.3 Multithreading Models

These models define the relationship between user threads and kernel threads.

1. Many-to-One Model - Many user-level threads are mapped to **one** kernel thread. - Thread management is done in user space → efficient. - **Drawback:** The entire process blocks if a single thread makes a blocking system call; also, only one thread can access the kernel at a time, so true parallelism on multiprocessors is not possible. - Example: early Green Threads, Solaris (early versions).

2. One-to-One Model - Each user-level thread is mapped to a **separate** kernel thread. - Allows multiple

threads to run in parallel on multiple processors. - Provides more concurrency than the many-to-one model. -

Drawback: Creating a user thread requires creating a corresponding kernel thread, which adds overhead; most implementations restrict the number of threads a process can create. - Example: Windows, Linux.

3. Many-to-Many Model - Multiplexes many user-level threads to a smaller or equal number of kernel threads. - The number of kernel threads may be specific to a particular application or machine. - Combines the best features of the above two models: developers can create as many user threads as needed, and the corresponding kernel threads can run in parallel on multiprocessors. - When a thread blocks, the kernel can schedule another thread for execution. - Example: older Solaris, some IRIX/Windows implementations.

Diagram description (for all three): - *Many-to-One*: Multiple user threads (U1, U2, U3) → all point to a single Kernel Thread (K1). - *One-to-One*: U1→K1, U2→K2, U3→K3 (each has its own kernel thread). - *Many-to-Many*: U1, U2, U3, U4 map to a smaller set, e.g., K1, K2 (multiplexed).

2.5 Process-Related Commands (Linux)

Command	Description
top	Displays real-time, dynamic view of running processes — shows PID, CPU%, memory%, running time, etc. Continuously updates.
ps	Displays a snapshot of currently running processes (static, one-time list). Common options: <code>ps -ef</code> (full listing), <code>ps aux</code> (all processes with details).
kill	Sends a signal to a process to terminate it. Syntax: <code>kill <PID></code> (default signal SIGTERM), <code>kill -9 <PID></code> (force kill, SIGKILL).
wait	Used by a parent process to pause execution until a child process completes. Ensures synchronization between parent and child.
sleep	Pauses/delays execution for a specified number of seconds. Syntax: <code>sleep 10</code> (pauses for 10 seconds).
exit	Terminates the current shell/process and returns control (with an optional exit status code) to the parent process.
nice	Starts a process with a modified scheduling priority. Lower “nice” values = higher priority. Syntax: <code>nice -n 10 command</code> (runs command with lower priority). Range typically: -20 (highest priority) to 19 (lowest priority).

Additional useful notes: - `ps` and `top` are used for **monitoring**, `kill` for **process control**, `nice` / `renice` for **priority management**. - `wait` is commonly used in shell scripts to ensure a background process finishes before the script continues. - Only the process owner or root/superuser can normally `kill` or `renice` a process (unless permissions allow otherwise).

Quick Revision Summary

- **Process** = program in execution; states: New, Ready, Running, Waiting, Terminated.
- **PCB** stores all process information (PID, PC, registers, memory info, etc.).
- **Schedulers**: Long-term (job pool → memory), Short-term (ready → CPU), Medium-term (swapping).
- **Context switch** = saving/restoring process state — pure overhead.
- **IPC**: Shared Memory (fast, manual sync) vs Message Passing (slower, kernel-managed, good for distributed systems).
- **Threads**: lightweight processes; benefits = responsiveness, resource sharing, economy, scalability.
- **Multithreading models**: Many-to-One, One-to-One, Many-to-Many.
- **Commands**: `top`, `ps`, `kill`, `wait`, `sleep`, `exit`, `nice` — used for process monitoring and control in Linux.